

=====

PlayList : spring boot full course
channel : india free internship
youtube channel <https://www.youtube.com/@IndiaFreeInternship>
source Code: [spring boot source Code](#)

=====

SESSION 1

- 1. SPRING FRAMEWORK**
 - 2. SPRING BOOT**
 - 3. SRPING DATA JPA**
 - 4. SPRING WEB MVC**
 - 5. SPRING REST**
 - 6. SPRING SECURITY**
 - 7. MICROSERVICE**
 - 8. SPRING BATCH**
 - 9. SPRING SCHEDULAR**
- =====

SESSION 2

=====

Technolgies : webPage(servlets) + Database(JDBC/JPA) + Email (Java Mail) + Message(JMS)

Limitations:

- Start coding from line zero
- Development takes more time
- More chance of error
- Design pattern applied manually

Framework:

- Ready-made structure
- In-built technologies
- Design patterns already implemented

RAD => Rapid Application Development

- Fast coding
- Less Error

Spring Framework:

- used for Web Application or Distributed appliation
- It has own Apis
- It Follow Container System

Spring Container

- Find/Scan Classes[spring bean]
- Create Object
- provide DATA
- Link Object based on Relation
- Finally Destroy

Dependency Injection (DI)

Dependency Injection is a concept/theory which is implemented by spring IoC (Inversion of control) also called as spring container.

Theory	programming
oops	core java
ORM	Hibernate with JPA
DI	Spring IoC(Inversion of control)

session 3

Application

3 Type of application

- (i) Web Application
- (ii) Distributed Application
- (iii) Microservices Application

(i) Web Application : Collection of web pages run under webserver.

Application : - n services

Project :- n Module

eg : Gmail Application

User(Register and Login) , Inbox, Sent, Drafts, Setting, etc

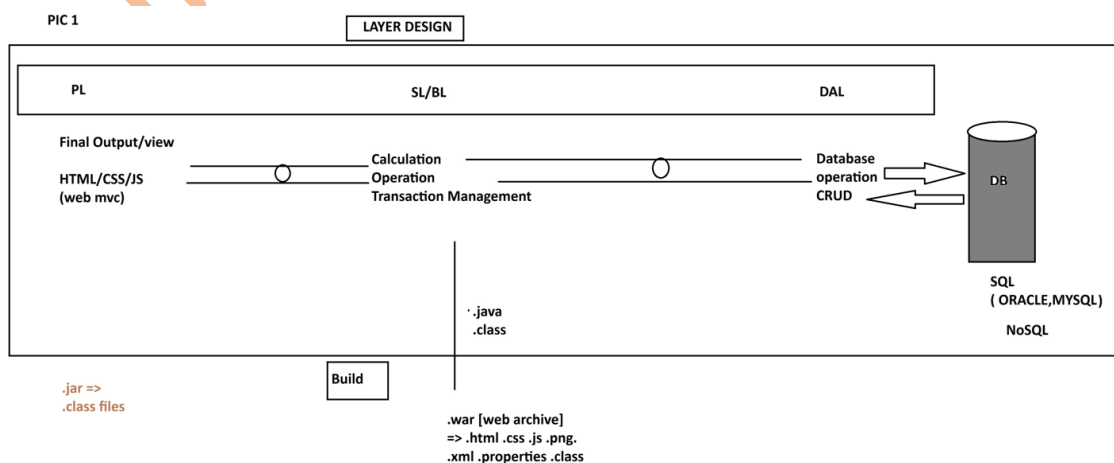
Amazon Application

User(Register and Login) , search, Cart, Payment, ...etc

=> Implementation of service / module: - This is implemented using 3 Layers Design.

1. DAL : Data Access Layer (Database Operations)
2. SL : Service Layer (Calculation/Operations/txn mngmt)
3. PL : Presentation layer (view /Dynamic web pages/MVC)

PIC 1



=====

(ii) Distributed Application : An Application that runs at mutiple devices is called as distributed appliation.

xyz Bank application

This appliation can be accessed in different ways. Example.

eg:

- 1. Manually Process**
- 2. NetBanking (Web App)**
- 3. Mobile Banking**
- 4. IVR**
- 5. ATM / DC**
- 6. 3rd party app (Gpay, PayPhone..etc)**

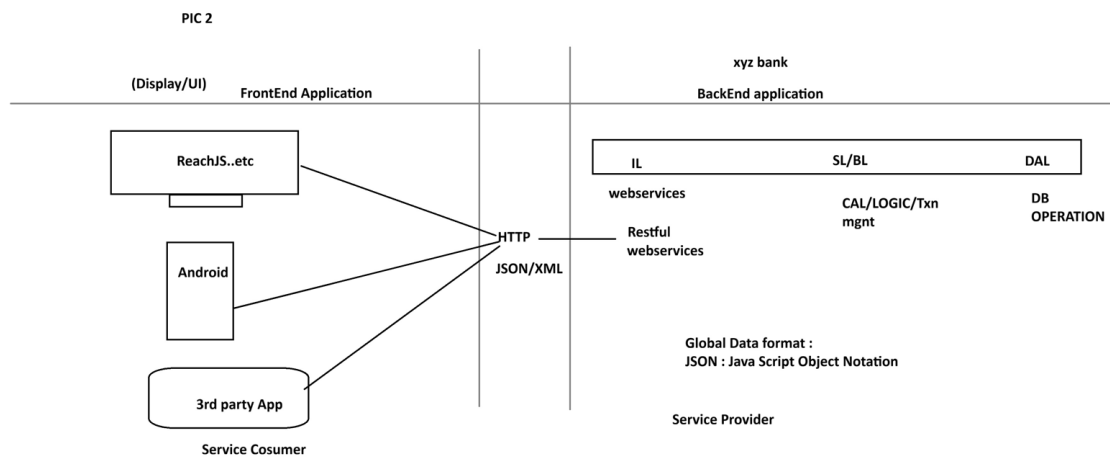
.....

-But here business logics exist at backend application.

- Fontend applications are implemented using Angular, Android...etc

- App connected using HTTP Protocol using Global Data Format (JSON/XML)

PIC 2

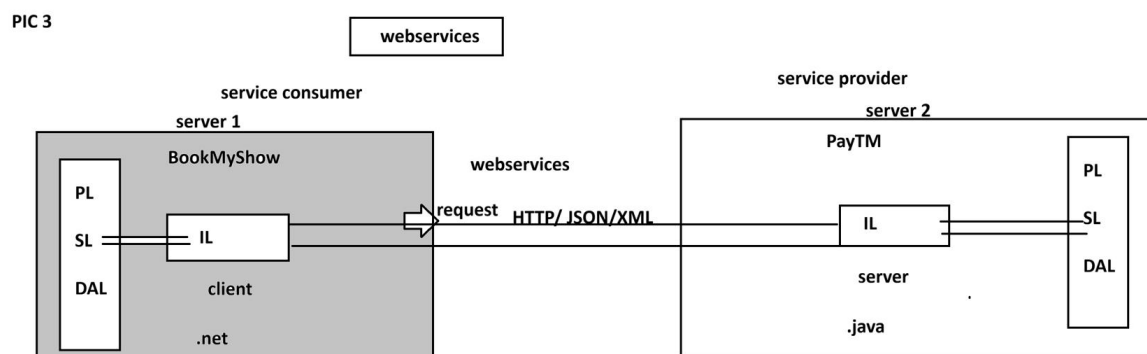


WebServices:

=====

Intergration of multiple (at least two) applications which runs on different servers and implemented using different language (java, .net, php, python....etc)

PIC 3



India Free Internship

session 4

=====

=====

SPRING FRAMEWORK

=====

CORE

1. DI and IOC
2. Container Types
3. Annotation Configuration
4. @ComponentScan
5. @Component
6. @Value
7. Java Configuration
8. @Configuration
9. @Bean
10. @PropertySource
11. Environment
12. Lifecycle methods
13. Runners
14. @ConfigurationProperties
15. Properties file
16. YAML File
17. Profiles
18. Lombok API

=====

spring core chapter 1

=====

Core:

This module/chapter gives all rules and guidelines to work with spring container.

1. DI and IOC

=====

Spring Container :- It takes care of object.

1. Find/Scan our Classes(spring bean)
2. Create Object
3. Provide Data
4. Link Object
5. Destroy the Object

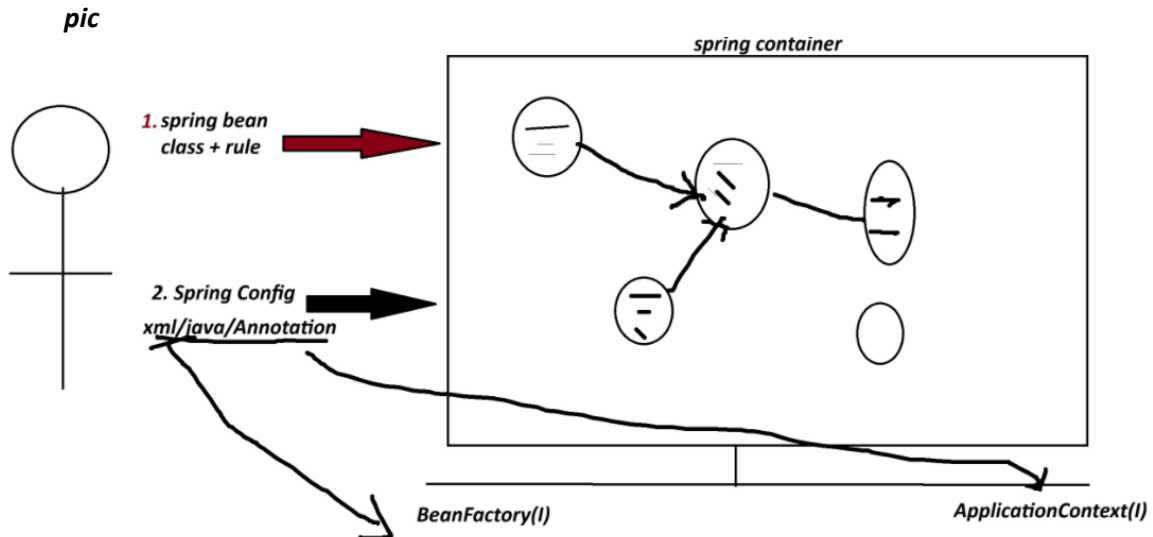
=>For this programmer has to provide two inputs-

1. Spring Bean : class+rules given by spring container.
2. Spring Configuration : This is main input contains details like object name, relation with objects, data....etc.

xml/java/Annotation

**** Spring Container 2 Types:**

1. BeanFactory (Old container) : support only XML configuration file.
2. ApplicationContext (New container) : support xml/java/Annotation configuration.



Dependency Injection (DI)

<i>Theory</i>	<i>programming</i>
<i>OOPs</i>	<i>java</i>
<i>ORM</i>	<i>Hibernate with JPA</i>
<i>DI</i>	<i>Spring IoC(Spring Container) -> Inversion of Control</i>

=> **Dependency** :- An instance variable (Field) exist inside a class (spring bean)

1. Primitive Type Dependency (PTD) :- If a variable is created using (8+1) datatype.
(byte, short, int, long, float, double, boolean, char +string)
(Their wrapper classess too)

2. Collection Type Dependency (CTD) :- If a variable created using(4) types:
(java.util.package)
(List,Map,Set and Properties)

3. Reference Type Dependency (RTD) :- If a variable is creatd using other class/interface

Injection(4)

=====

=> It give/provide/assign existed provide data to Dependency(Inside object)

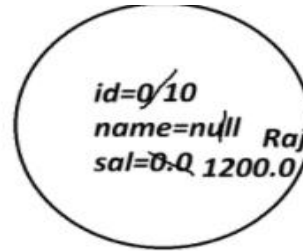
1. Setter Injection (SI) :- provide data to variable using its set method(setter).

2. Constructor Injection(CI): provide data creating object using parameterized constructor.

3. LookUpMethod Injection(LMI) :- Spring bean Scope
[When parent bean is sigleton and child bean is prototype]

4. Interface Injection(II) : - Not Exist in Spring F/W.

```
class Employee{
    int id;
    String name;
    double sal;
}
```



Employee e=new Employee()

```
class Product{
    int id; //PTD
    String name; //PTD
    Boolean exist; //PTD
```

```
class Vendor{}

interface Service{}
```

```
List<String> models; //CTD
Map<String,Integer> data; //CTD
```

```
Vendor vob; //RTD
Service sob; //RTD
```

```
}
```

SESSION 5

=====

1. Spring Bean

2. STS Installation

Spring Bean

=====

-It is class that follow rules given by Spring Container.

Rule

- class must be public Type.
- Recomended to keep in a package (basePackage Rule)

ex :

com.app.service

com.app.controller

- Fields(Variables) are option, if required recomended to keep private.

```
private Integer empId;
```

```
private String empName;
```

- If variable present in class,

=>define default constructor with setter/getter

=>(or/and) Parameterized Constructor.

ex:

```
public class Employee{  
    private Integer id;  
    private String name;  
    public Employee(){  
    }  
    public void setId(Integer id){  
        this.id=id;  
    }  
  
    public Integer getId(){  
        return id;  
    }  
}
```

e. We can override Object(c) methods:

toString() equals() hashCode()

[because non-private, non-final,non-static]

f. Inheritance Rule:

Allowed ->Only Spring API is allowed (ie spring F/W classes and interface)

not allowed-> EJB, Servlet

g. Annotation Rule:

Only we can add Spring API Annotation and Inetegration API Annotations

(Hibernate with JPA, Restful webServices annotations);

ex:

@Component, @Service, @Repository.. etc.

STS Installation

=====

step 1:

go to official website

spring.io

link : - <https://spring.io/tools#eclipse>

step 2 download STS according your OS

step 3

Rigth click on ZIP file

click on "Extract Here"

step 4

Open extracted folder

Run : SpringToolsForEclipse

Final STEP STS installed successfully.

=====

session 6

=====

1. Spring Bean

Simple Java class that follow rules given by container.

2. Spring Configuration File(XML)

It gives object

details (name, data, links with other objects ... etc).

3. Test class=> Just find, container created or not? object is created or not?

=====

Spring core XML Tags

=====

<bean> - Object creation

<property> - calling set method

<constructor-arg> - call parameterized constructor

<value> - To provide data to Primitive Type.

<list> <set> <map> <props> -- To provide data to Collection type.

<ref/> -- To Link two objects

1. To create Object

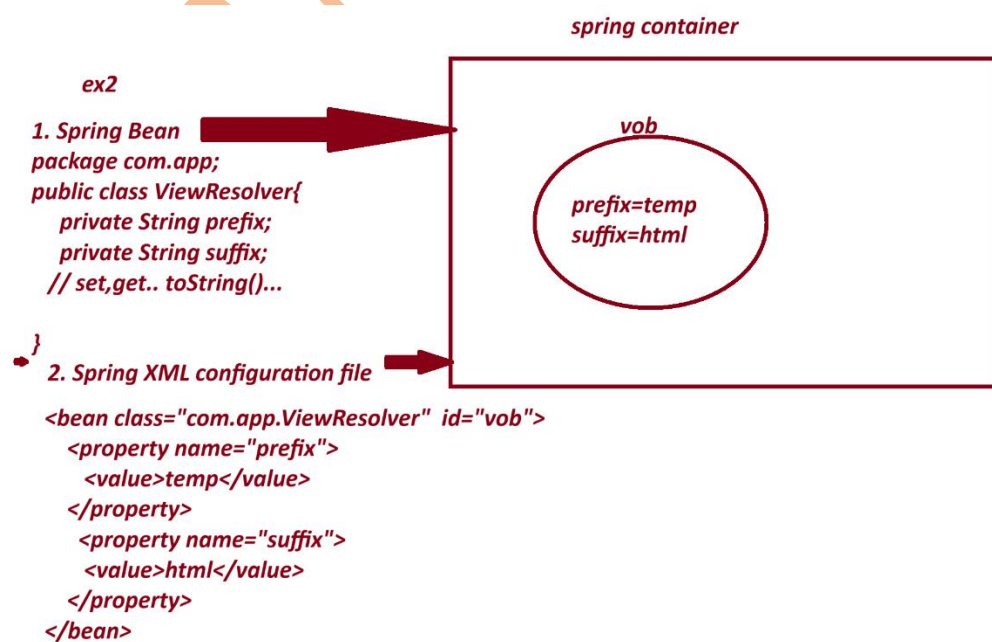
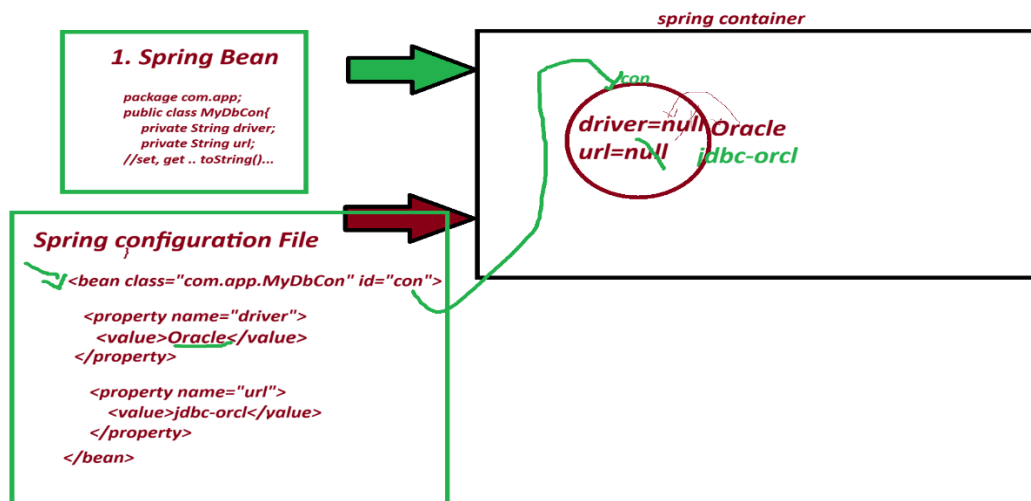
<bean id="objName" class="FullyQualifiedClassName"></bean>

2. To call set method

<property name="variableName"></property>

3. To provide data to Primitive Variables

<value>data</value>



1. Spring bean

package com.app;

```
public class PaymentService{  
  
    private int token;  
  
    private boolean active;  
  
    private String provider;  
  
    //set.. set toString..etc  
}
```

2. Spring XML configuration file.

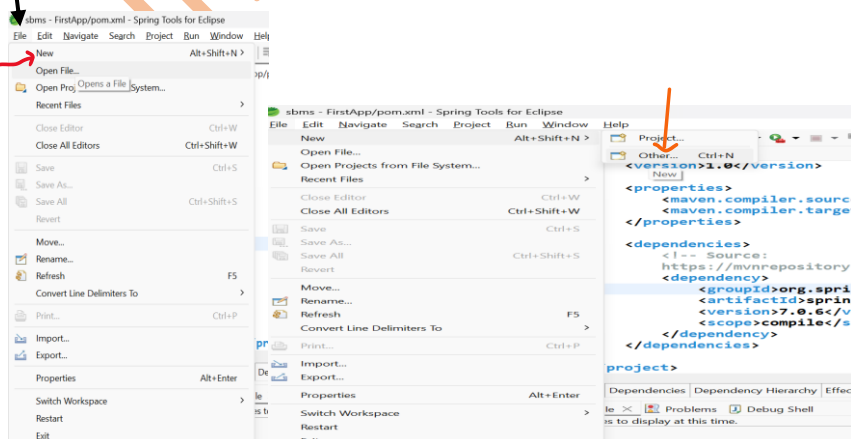
```
<bean class="com.app.PaymentService" id="psObj">  
  
    <property name="token">  
  
        <value>154678</value>  
  
    </property>  
  
    <property name="active">  
  
        <value>true</value>  
  
    </property>  
  
    <property name="provider">  
  
        <value>xyzBank</value>  
  
    </property>  
  
</bean>
```

=====

CREATE MAVEN PROJECT WITH BELOW STEP :-

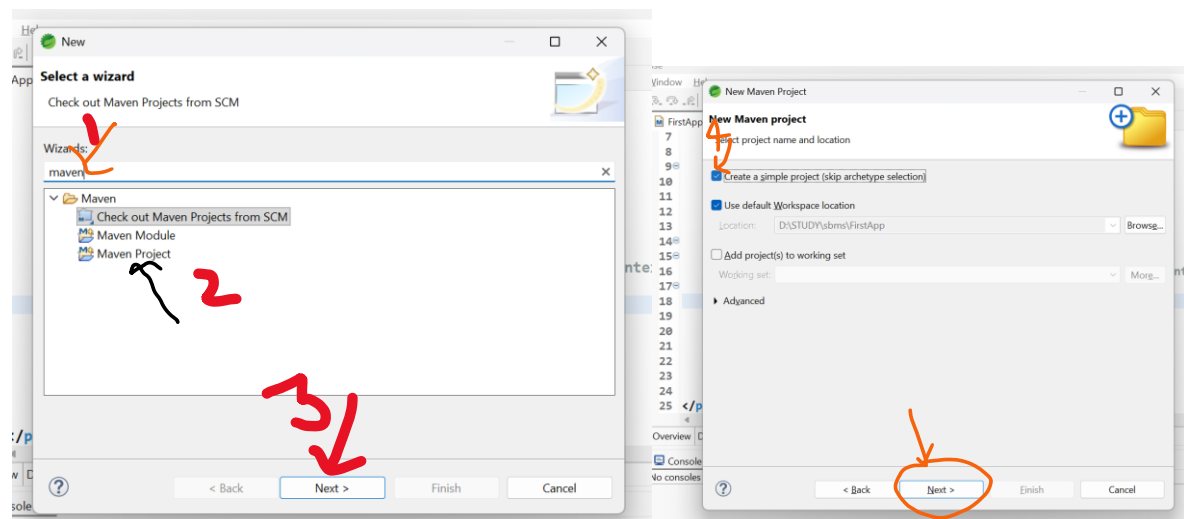
=====

1. Create Project



File-> New-> Other-> Type maven->choose maven project-> next->

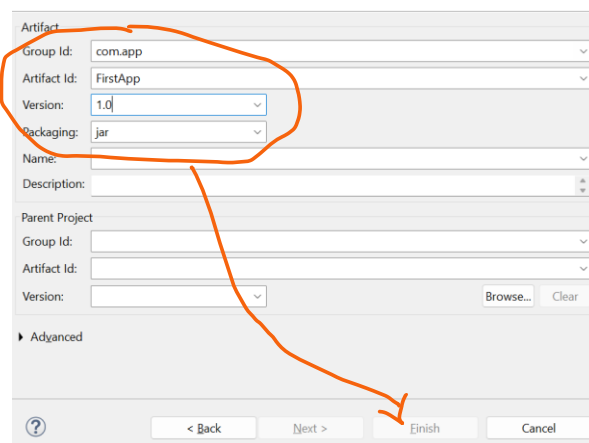
choose checkbox [v] create simple project -> next-> Enter details



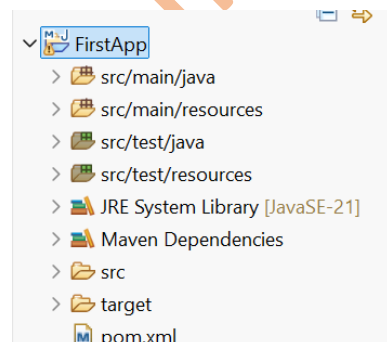
groupId : com.app

ArtifactId : FirstApp

version : 1.0



-> Finish



2. pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>

<artifactId>FirstApp</artifactId>

<version>1.0</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

</properties>

<dependencies>

    <!-- Source:
        https://mvnrepository.com/artifact/org.springframework/spring-context -->

    <dependency>

        <groupId>org.springframework</groupId>

        <artifactId>spring-context</artifactId>

        <version>7.0.6</version>

        <scope>compile</scope>

    </dependency>

</dependencies>

</project>

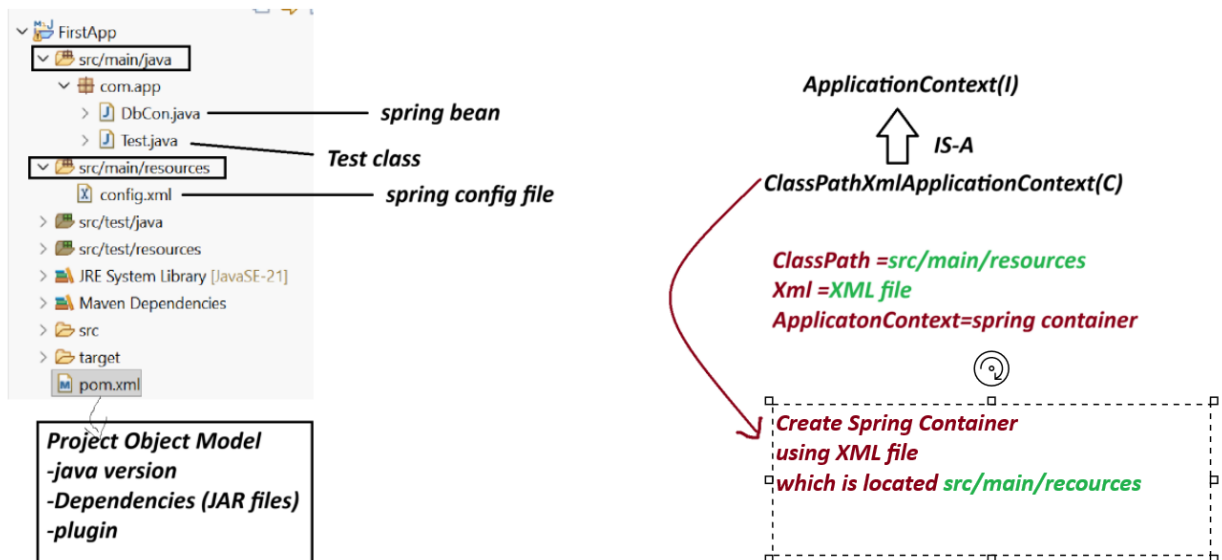
```

session 7

=====

Spring Core First Application 3 Files.

1. Spring bean (.java)
2. Spring XML configuration (config.xml)
3. Test class(.java)



Note:-

1. Spring container are two Types.

(a) Legacy Container : BeanFactory(I) support only XML

(b) New Container : ApplicationContext(i) support all XML/java/Annotation config.

(c) ApplicationContext(I) : it is a interface.

we are using one of its Impl class:

ClassPathXmlApplicationContext(C).

Create Maven Project with below Step:

File>new>other>type maven>choose maven project>next>choose checkbox> create Simple project>next>Enter details

GroupId - com.app

ArtificatId - SecondApp

version - 1.0

code

SecondApp

pom.xml

=====

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>
<artifactId>SecondApp</artifactId>
<version>1.0</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>
</properties>

<dependencies>
<!-- Source:https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>7.0.5</version>
<scope>compile</scope>
</dependency>
</dependencies>
</project>
```

DbCon.java

```
package com.app;
```

```
public class DbCon {
    private String driver;
    private String url;
```

```

//alt+shit+s

public DbCon() {
super();
System.out.println("Constructor called");
}

public String getDriver() {
return driver;
}

public void setDriver(String driver) {
this.driver = driver;
System.out.println("Driver method called");
}

public String getUrl() {
return url;
}

public void setUrl(String url) {
this.url = url;
System.out.println("Url method called");
}

@Override
public String toString() {
return "DbCon [driver=" + driver + ", url=" + url + "];"
}

}

```

=====

config.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```

```
xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean class="com.app.DbCon" id="mobj">
    <property name="driver">
        <value>Oracle Driver</value>
    </property>

    <property name="url">
        <value>JDBC-ORCL</value>
    </property>
</bean>
```

```
</beans>
```

```
=====
```

Test.java

```
-----
```

```
package com.app;
```

```
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
public class Test {
```

```
//ctrl+shit+O
```

```
public static void main(String[] args) {
```

```
    ApplicationContext ac=new ClassPathXmlApplicationContext("config.xml");
```

```
    Object ob = ac.getBean("mobj");
```

```
    System.out.println(ob);
```

```
}
```

```
}
```

output

Constructor called

Driver method called

Url method called

DbCon [driver=Oracle Driver, url=JDBC-ORCL]

Session 8

=====

India Free Internship